LECTURE 4
STACK

AGENDA

4.1 What is a Stack?

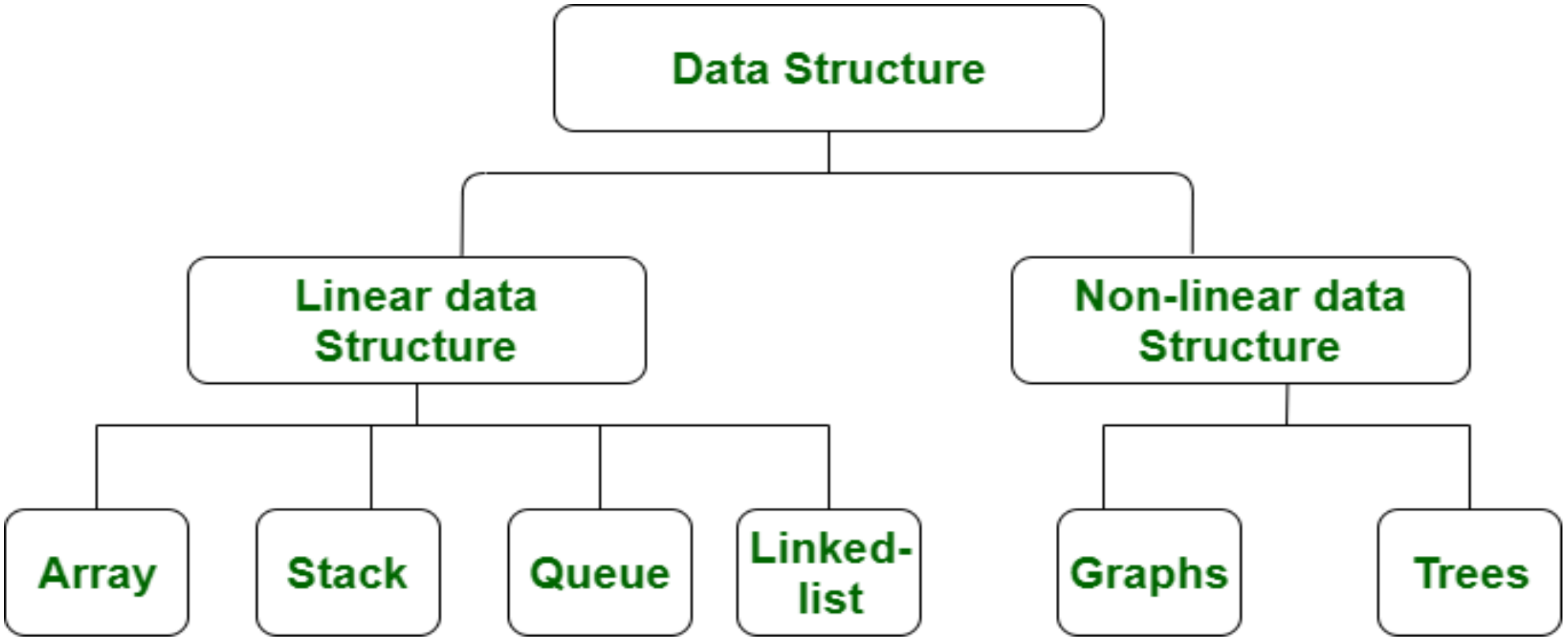
4.2 Stack Operations

4.3 Stack Implementation as an Array

4.4 Stack Implementation as a Linked List


REMEMBER
!

CLASSIFICATION OF DATA STRUCTURE



STACK



BASIC IDEA

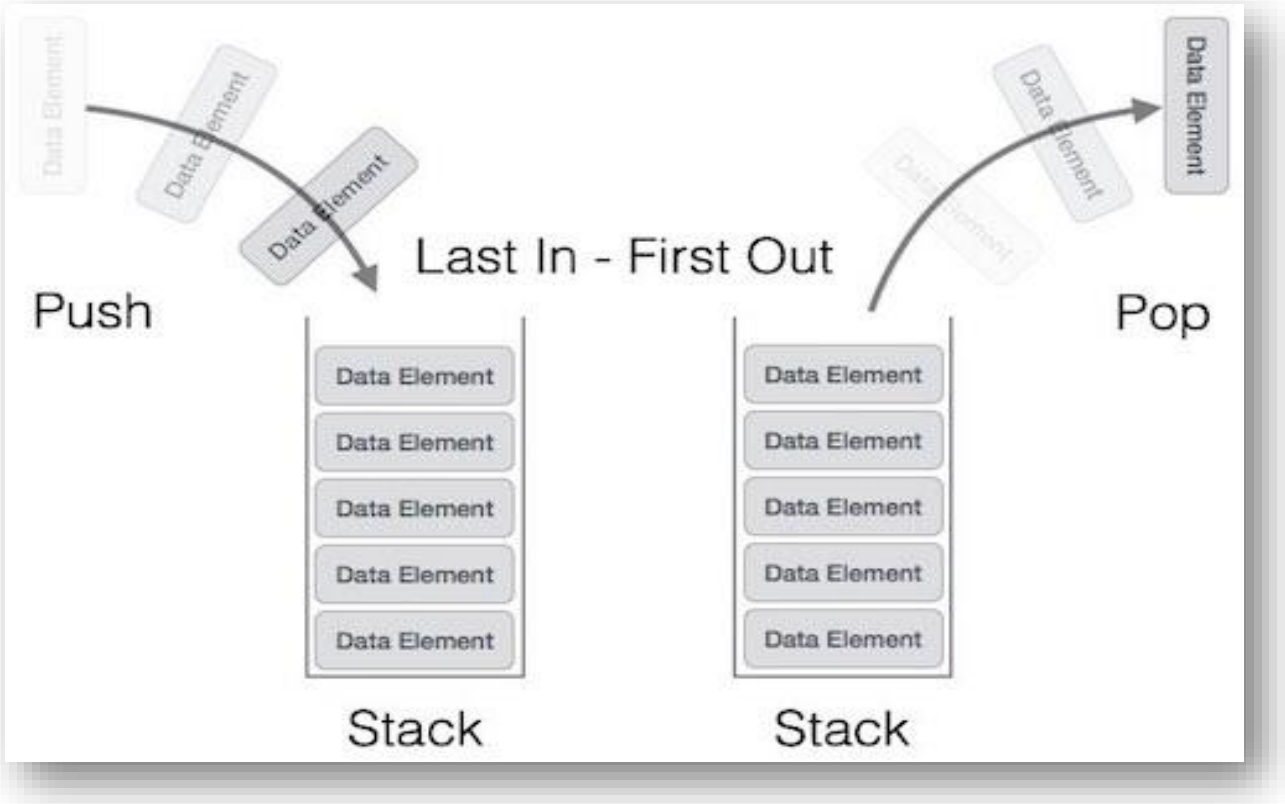
A stack is an Abstract Data Type (ADT), commonly used in most programming languages. It is named stack as it behaves like a real-world stack, for example – a deck of cards or a pile of plates, etc.



4.1. WHAT IS A STACK?

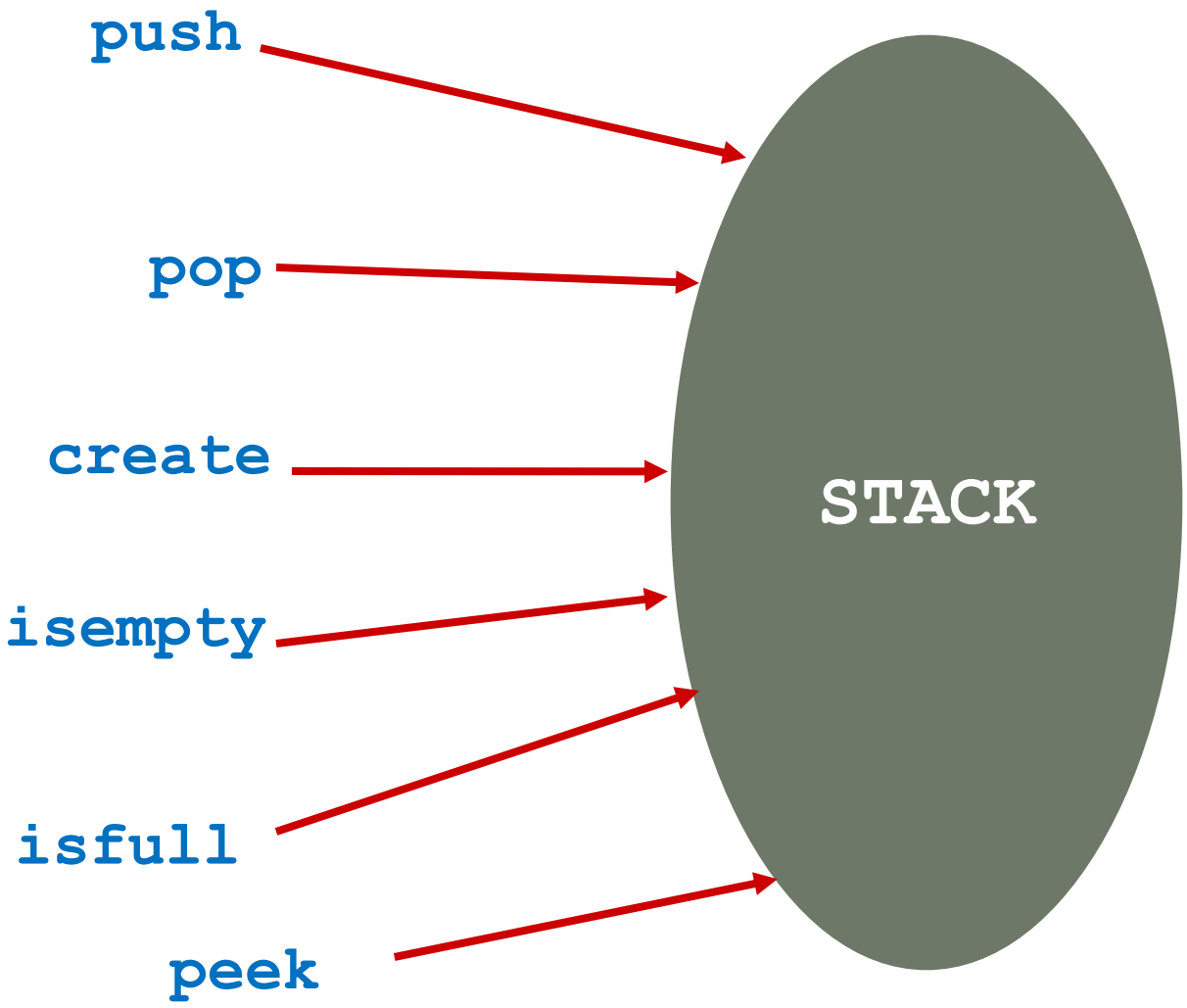
- A stack in data structures is a **linear** collection of elements that follows a particular order in which the operations of addition and removal of elements are performed.
- The order is referred to as "**Last In, First Out**" (**LIFO**). This means that the last element added to the stack is the first one to be removed.
- Stacks are crucial for managing **function calls** in almost all programming environments, helping maintain the order of operations and execution seamlessly.

STACK REPRESENTATION



- Can be implemented by means of Array, Structure, Pointers and Linked List.
- Stack can either be a fixed size or dynamic.

4.2. STACK OPERATIONS



4.2. STACK OPERATIONS (CONT.)

- As stack is a linear data structure.
- It is controlled by two **main** operations: **push** and **pop**.
- Both the operations take place from one end of the stack,
- usually called top. Top points to the current top position of the
- stack.
- Push operation adds an element to the top of the stack.
- Pop operation removes an element from the top of the stack.
- These two operations implements the LIFO method .

STACK IMPLEMENTATION

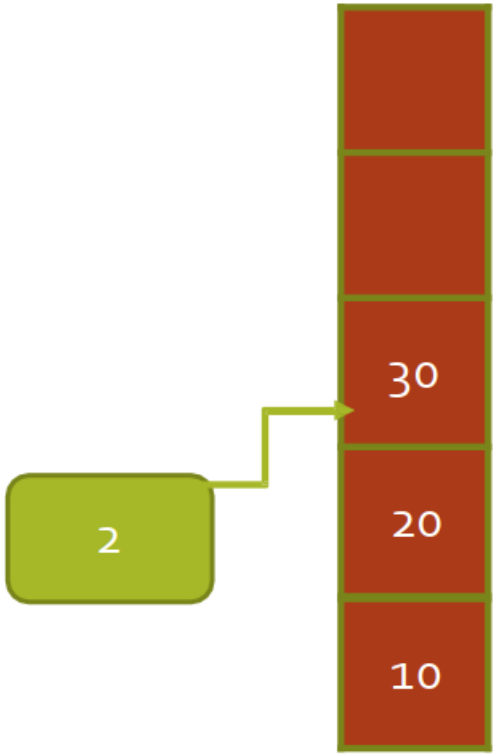
- Stack can be implemented in two ways:
  - As an Array
  - As a Linked List

4.3.STACK USING ARRAY

STACK AS AN ARRAY

- Array implementation of stack uses :
  - an array to store data
  - an integer type variable usually called the top, which points to the top of the stack

NOTE: The variable top contains the index number of the top most filled element of the array.



STACK IMPLEMENTATION

```
1   #include <iostream>
2   using namespace std;
3
4   const int MAX = 5;  // Maximum size of the stack
5
6   class Stack {
7   private:
8       int arr[MAX];  // Array to store stack elements
9       int top;       // Index of the top element
10
11  public:
12      Stack() {
13          top = -1;  // Stack is initially empty
14      }
```

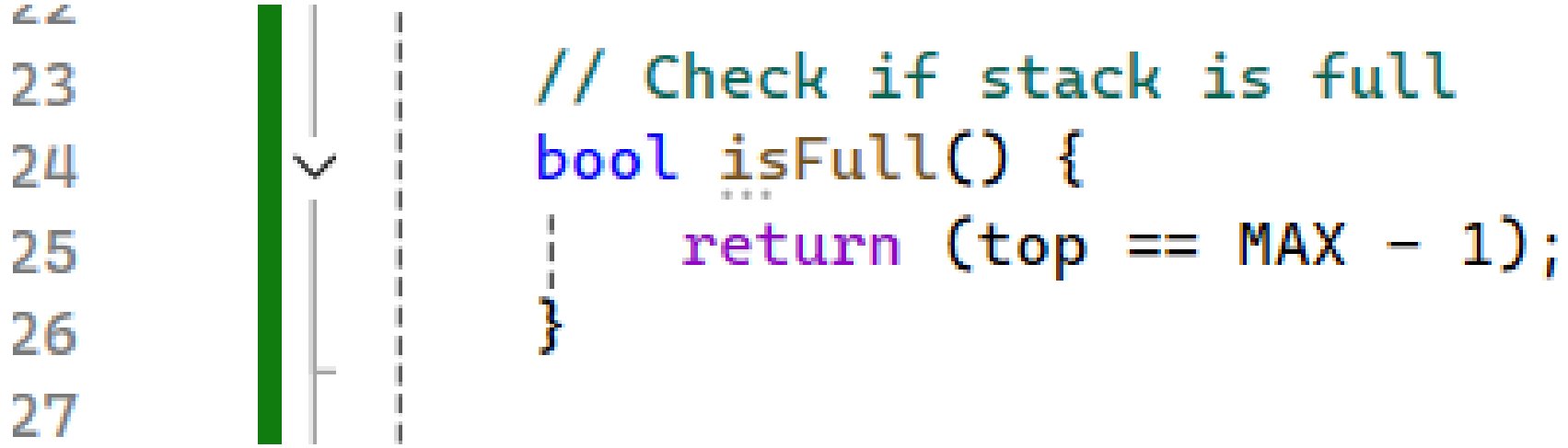
PUSH OPERATION

- Initially when the stack is empty, TOP can have any integer value other than any valid index number of the array. Let TOP =-1;
- The first element in the empty stack goes to the 0th position of the array and the Top is initialized to value 0.
- After this every push operation increase the TOP by 1 and inserts new data at that particular position.
- As arrays are fixed in length, elements can not be inserted beyond the
- maximum size of the array. Pushing data beyond the maximum size of the stack results in “data overflow”.

PUSH IMPLEMENTATION

```
28
29      // Function to add an element to the stack
30      void push(int value) {
31          if (isFull()) {
32              cout << "Stack Overflow! Cannot push " << value << endl;
33          }
34          else {
35              arr[++top] = value;
36              cout << value << " pushed into stack." << endl;
37          }
38      }
```

ISFULL() IMPLEMENTATION



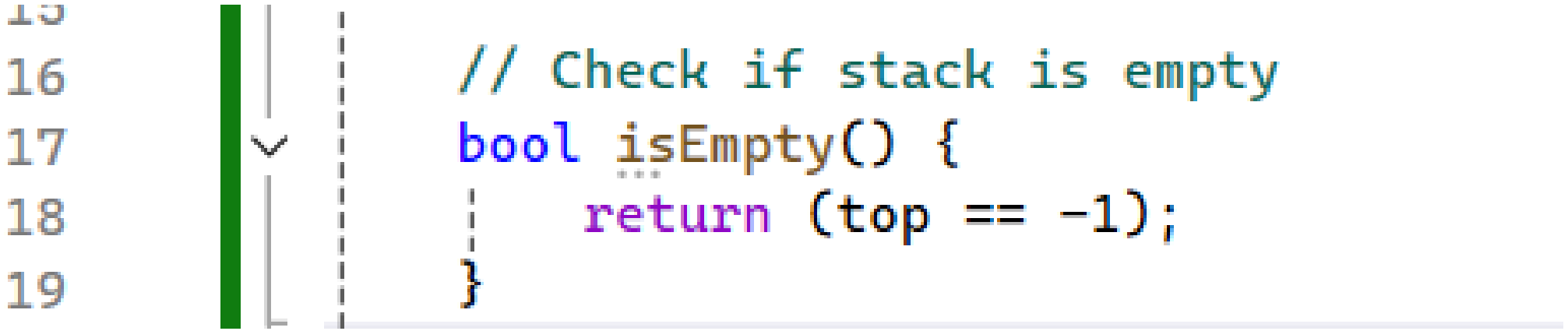
POP OPERATION

- The pop operation deletes the most recently entered item from the stack.
- Any attempt to delete an element from the empty stack results in “data underflow” condition.
- The variable TOP contains the index number of the current top position of the stack.
- Each time the pop operation is performed, the TOP variable is decremented by 1.

POP IMPLEMENTATION

```
39
40      // Function to remove the top element
41      void pop() {
42          if (isEmpty()) {
43              cout << "Stack Underflow! Cannot pop." << endl;
44          }
45          else {
46              cout << arr[top--] << " popped from stack." << endl;
47          }
48      }
```

ISEMPTY() IMPLEMENTATION



PEEK IMPLEMENTATION

```
50      // Function to get the top element
51      void peek() {
52          if (isEmpty()) {
53              cout << "Stack is empty." << endl;
54          }
55          else {
56              cout << "Top element is: " << arr[top] << endl;
57          }
58      }
```

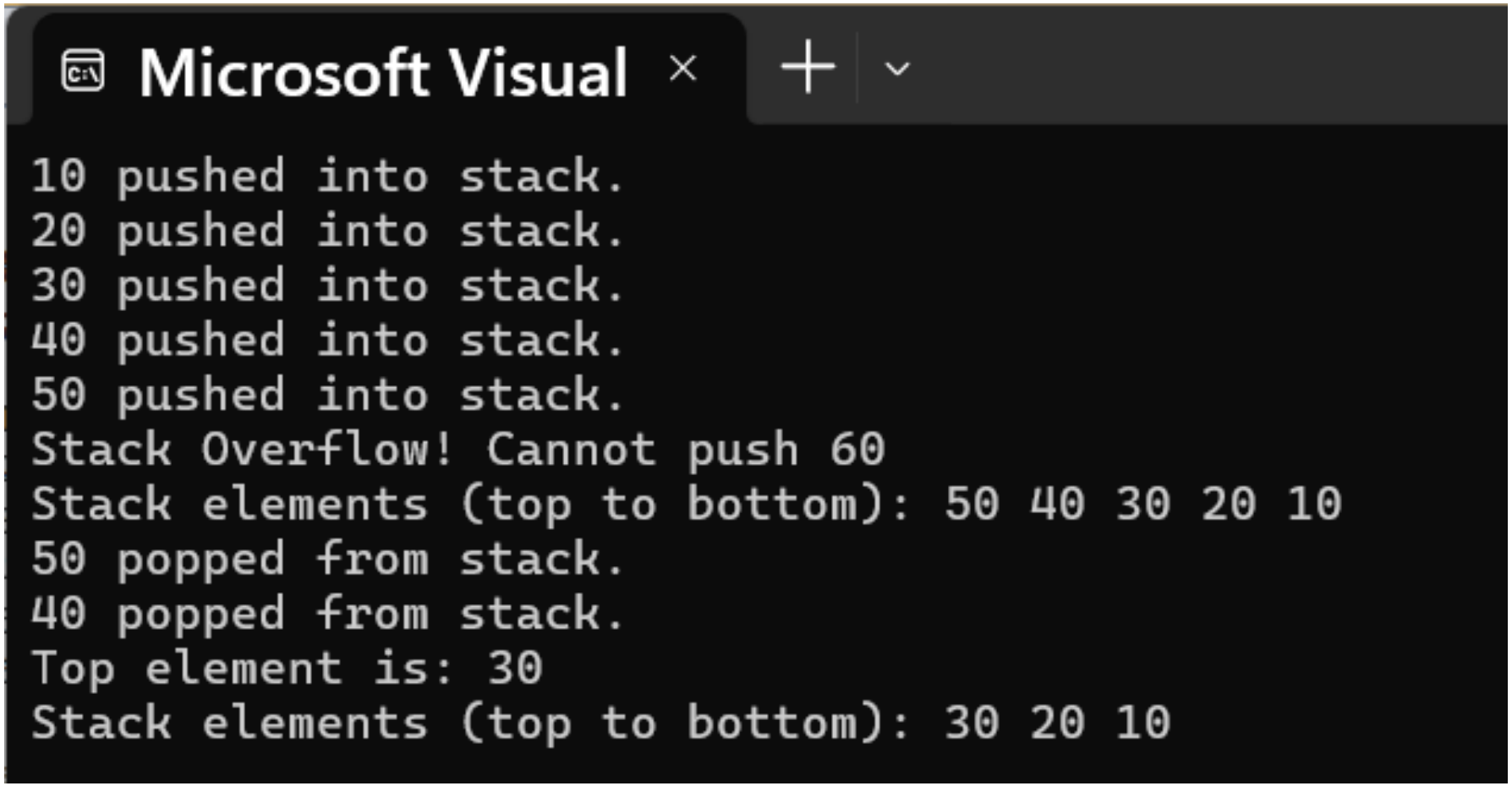
DISPLAY IMPLEMENTATION

```
61      // Display all elements in stack
62      void display() {
63          if (isEmpty()) {
64              cout << "Stack is empty." << endl;
65          }
66          else {
67              cout << "Stack elements (top to bottom): ";
68              for (int i = top; i >= 0; i--) {
69                  cout << arr[i] << " ";
70              }
71              cout << endl;
72          }
73      }
74  };
```

USAGE “MAIN()”

```
76  int main() {
77      Stack s;
78
79      s.push(10);
80      s.push(20);
81      s.push(30);
82      s.push(40);
83      s.push(50);
84      s.push(60);   // Overflow
85
86      s.display();
87
88      s.pop();
89      s.pop();
90
91      s.peek();
92      s.display();
93
94      return 0;
95  }
```

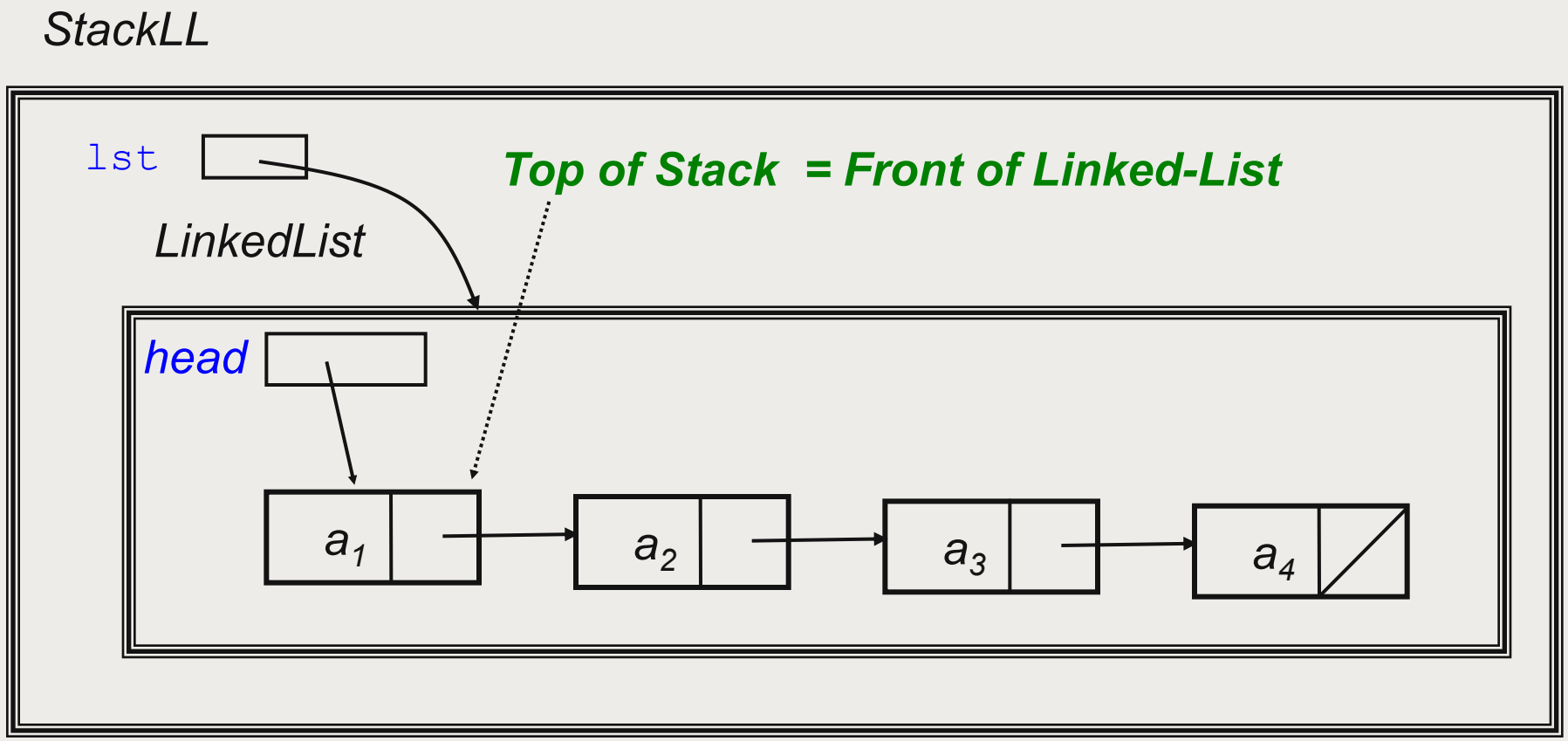
USAGE “MAIN()” (CONT.)



4.4. STACK USING LINKED LIST

IMPLEMENTATION BY LINKED LISTS

Can use a Linked List as implementation of stack



STACK IMPLEMENTATION

```cpp
#include <iostream>
using namespace std;

// Node structure
template <class T>
class Node {
public:
    T data;
    Node* next;
    Node(T value) {
        data = value;
        next = nullptr;
    }
};
```

STACK IMPLEMENTATION (CONT.)

```
16  // Stack class using Linked List
17  template <class T>
18  class Stack {
19  private:
20      Node<T>* top;  // Pointer to top node
21
22  public:
23      Stack() {
24          top = nullptr;
25      }
```

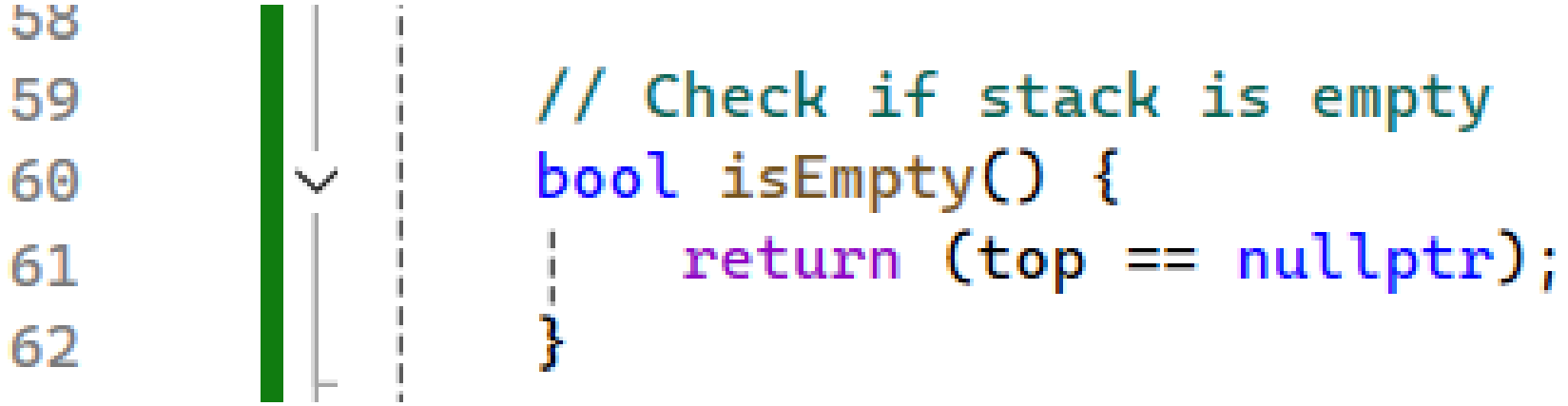
PUSH IMPLEMENTATION

```
27      // Push operation
28      void push(T value) {
29          Node<T>* newNode = new Node<T>(value);
30          newNode->next = top;
31          top = newNode;
32          cout << value << " pushed into stack." << endl;
33      }
```

POP IMPLEMENTATION

```
35      // Pop operation
36      void pop() {
37          if (isEmpty()) {
38              cout << "Stack Underflow! Cannot pop." << endl;
39          }
40          else {
41              cout << top->data << " popped from stack." << endl;
42              Node<T>* temp = top;
43              top = top->next;
44              delete temp;
45          }
46      }
```

ISEMPTY() IMPLEMENTATION



PEEK IMPLEMENTATION

```
49      // Peek (top element)
50      void peek() {
51          if (isEmpty()) {
52              cout << "Stack is empty." << endl;
53          }
54          else {
55              cout << "Top element is: " << top->data << endl;
56          }
57      }
```

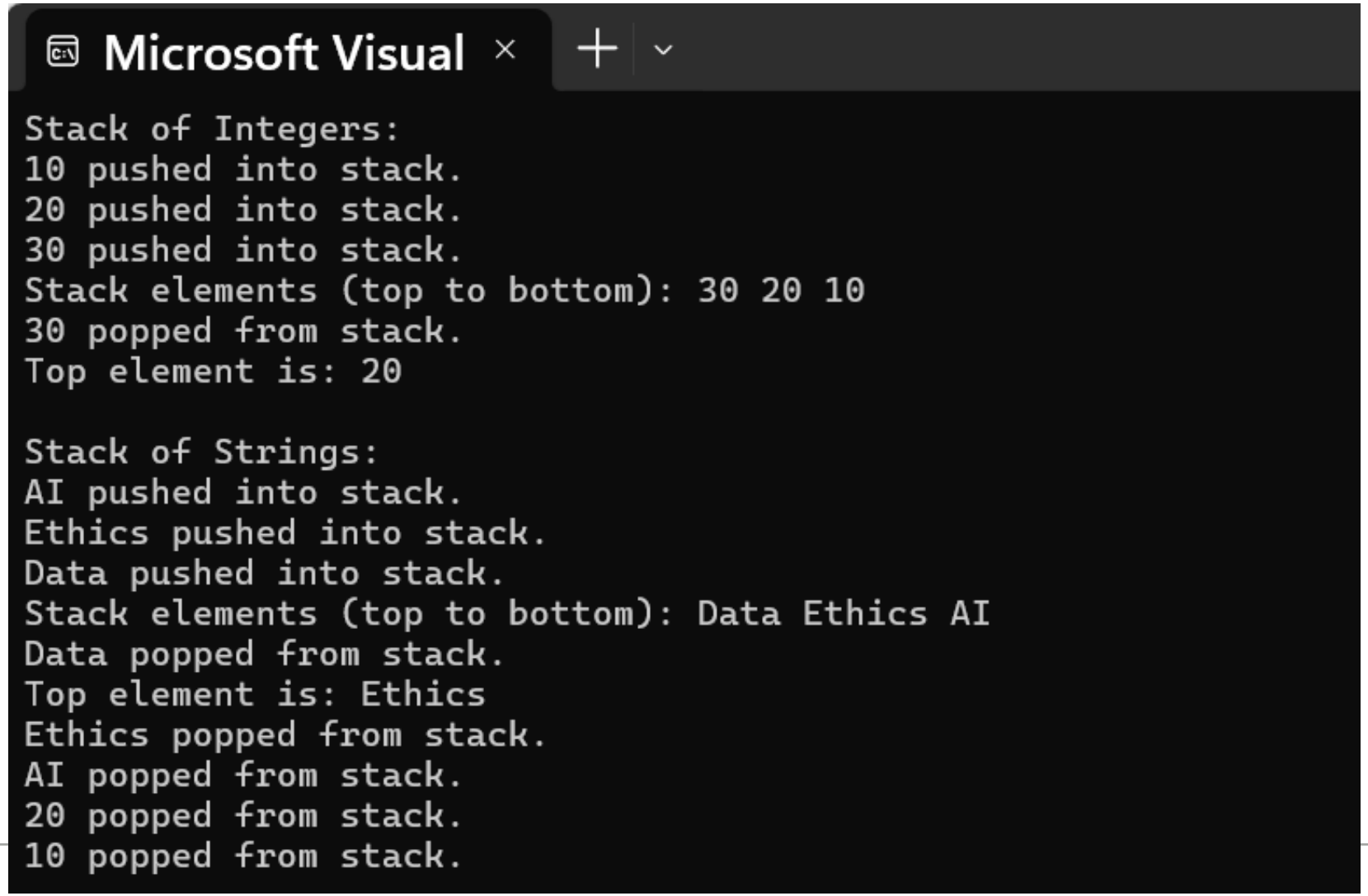
DISPLAY IMPLEMENTATION

```
64      // Display all elements
65      void display() {
66          if (isEmpty()) {
67              cout << "Stack is empty." << endl;
68          }
69          else {
70              cout << "Stack elements (top to bottom): ";
71              Node<T>* temp = top;
72              while (temp != nullptr) {
73                  cout << temp->data << " ";
74                  temp = temp->next;
75              }
76              cout << endl;
77          }
78      }
```

USAGE “MAIN()”

```
88  int main() {
89      cout << "Stack of Integers:\n";
90      Stack<int> s1;
91      s1.push(10);
92      s1.push(20);
93      s1.push(30);
94      s1.display();
95      s1.pop();
96      s1.peek();
97
98      cout << "\nStack of Strings:\n";
99      Stack<string> s2;
100     s2.push("AI");
101     s2.push("Ethics");
102     s2.push("Data");
103     s2.display();
104     s2.pop();
105     s2.peek();
106
107     return 0;
108 }
```

USAGE “MAIN()” (CONT.)



THANKS